

Curso de Especialización

Ciberseguridad en las TIC

Actividad 20. XSS

Resumen

Autora: Marina del Pilar López Oliva

Abril 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	22/04/2026	Realización de la práctica.
1.1	23/04/2026	Documentación de la práctica.

Índice de Contenidos

1. Room TryHackMe.....	4
2. Los seis retos XSS.....	6
Level 1 - XSS reflected basic.....	6
Level 2 - XSS almacenado en <blockquote>.....	7
Level 3 - XSS basado en el hash de la URL.....	9
Figura 15. Reto superado.....	11
Level 4 – XSS a través del parámetro de consulta con timer.....	11
Level 5 – XSS a través de enlace en parámetro "next".....	13
Level 6 – XSS a través de esquemas de URL no estándar.....	15
Todos los niveles completados.....	17
3. Funcionamiento del DOM Based XSS.....	17

1. Room TryHackMe

En este apartado, voy a incluir algunas capturas interesantes de las preguntas de TryHackMe.

Se accede al laboratorio en la dirección 10.128.144.249:3923, introduciendo el XSS que es vulnerable, lo de detrás del “?”. En el primer laboratorio, se probó el payload `<script>alert(1)</script>` en el parámetro de búsqueda, obteniendo como respuesta del sistema la cadena `?h#cc`.

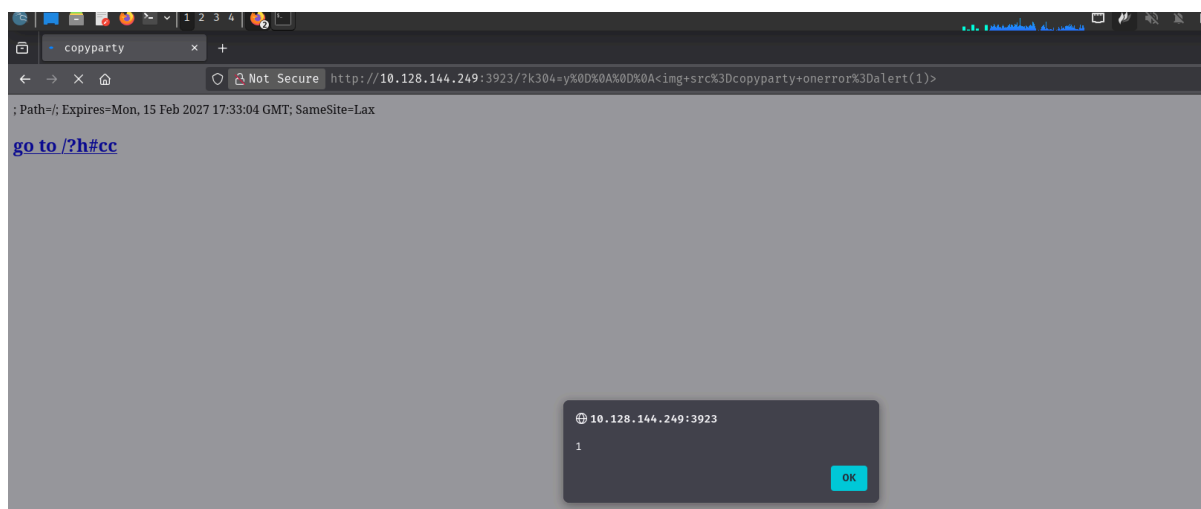


Figura 1. Acceso al primer laboratorio con la primera vulnerabilidad XSS.

Se explota otra vulnerabilidad XSS reflejado en el formulario de contacto, inyectando “`<script>alert(document.cookie)</script>`” y enviamos el mensaje.

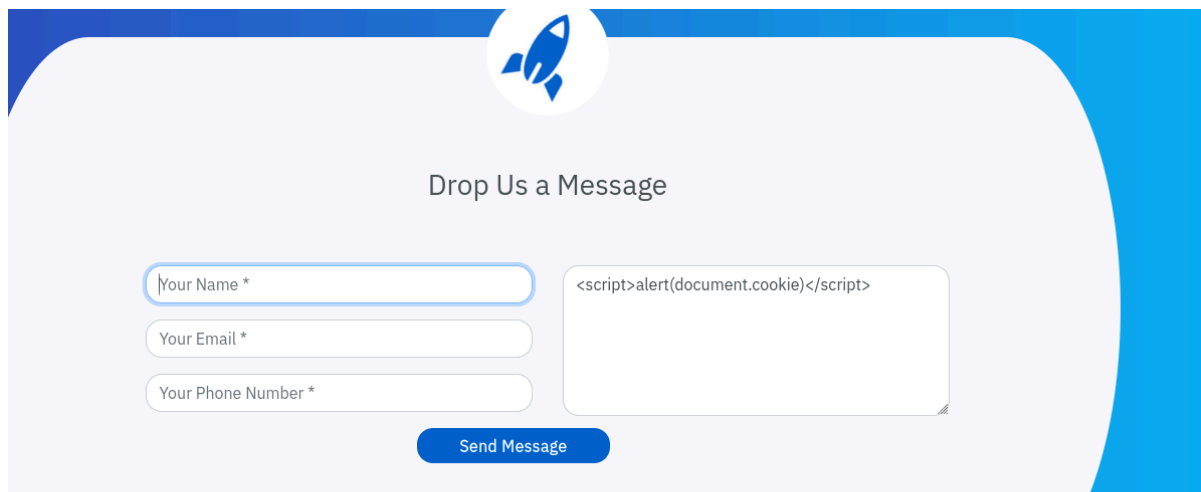


Figura 2. Explotación del segundo laboratorio con la segunda vulnerabilidad XSS.

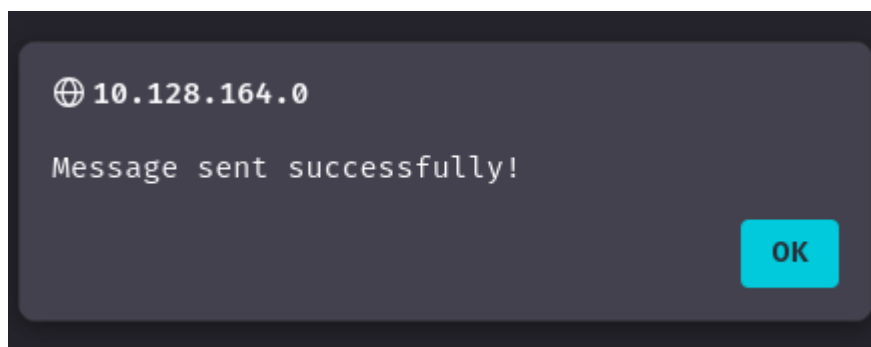


Figura 3. Mensaje enviado correctamente.

Una vez enviado el mensaje, vamos a loguearnos con el usuario “admin” con contraseña “admin123” en el apartado “Receptionist”.

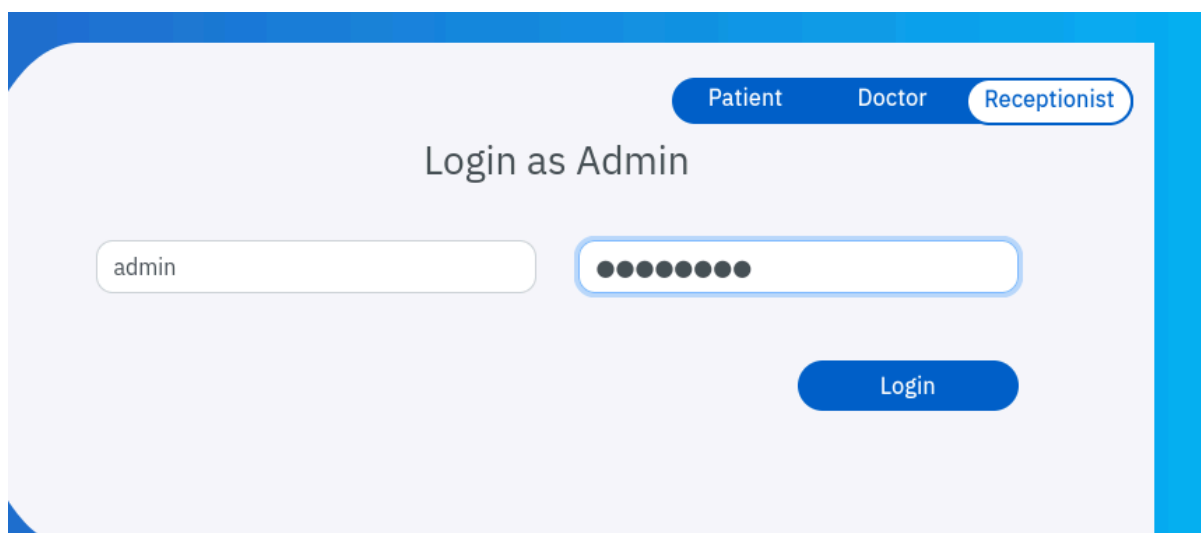


Figura 4. Inicio de sesión con el usuario Receptionist.

Y obtenemos la cookie PHPSESSID=pfg3j0bf8002mg1c6rvudh1mq4 .

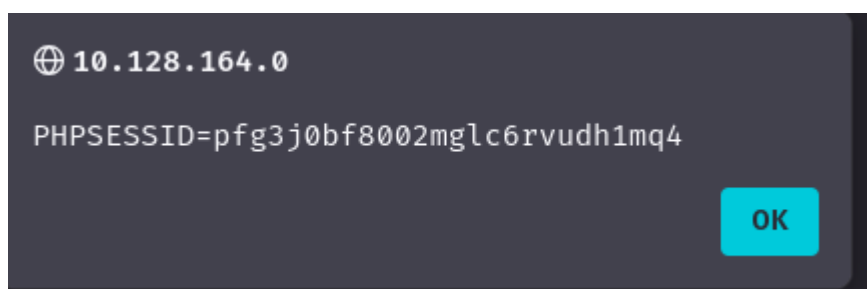


Figura 5. Cookie obtenida por la explotación del XSS.

Se puede observar que el perfil marlopoli, mi perfil de TryHackMe, que se ha completado el reto correspondiente a XSS, como se manda en esta actividad.

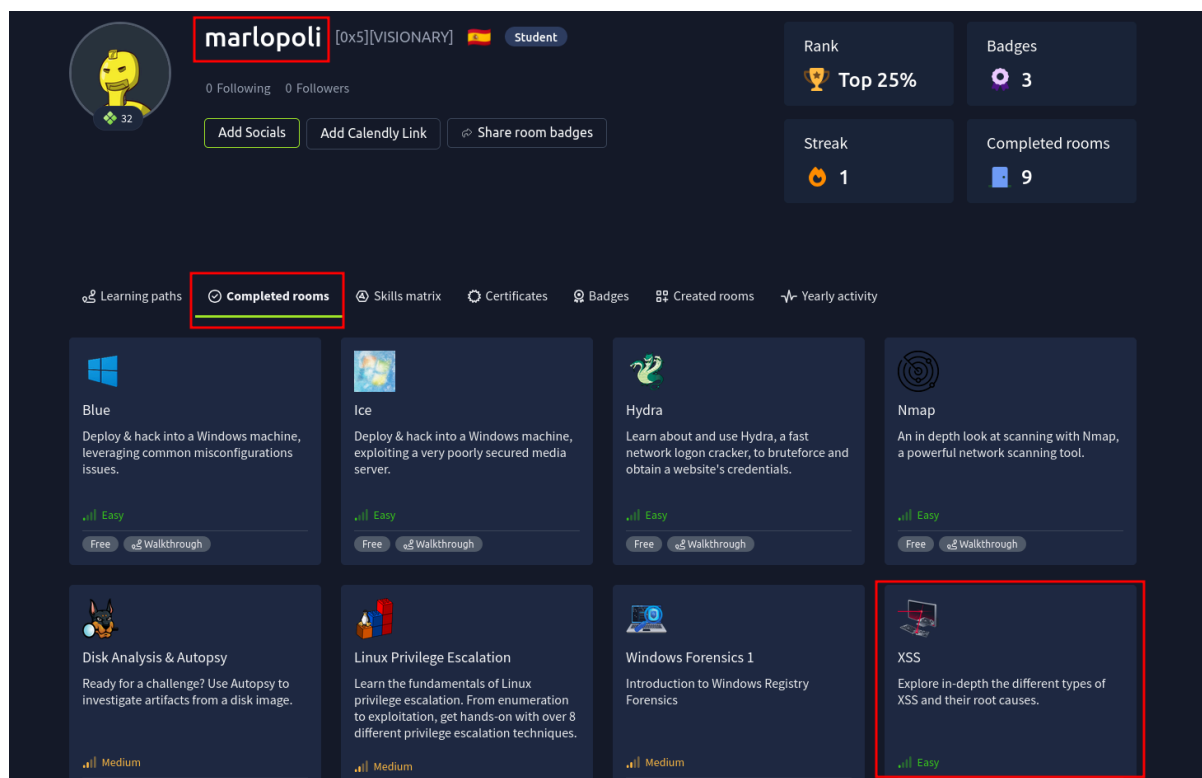


Figura 6. Laboratorio en TryHackMe completado.

2. Los seis retos XSS

Level 1 - XSS reflected basic

Como podemos ver en la imagen de abajo, el parámetro query se concatena directamente en el HTML sin ninguna sanitización. No hay escape de caracteres.

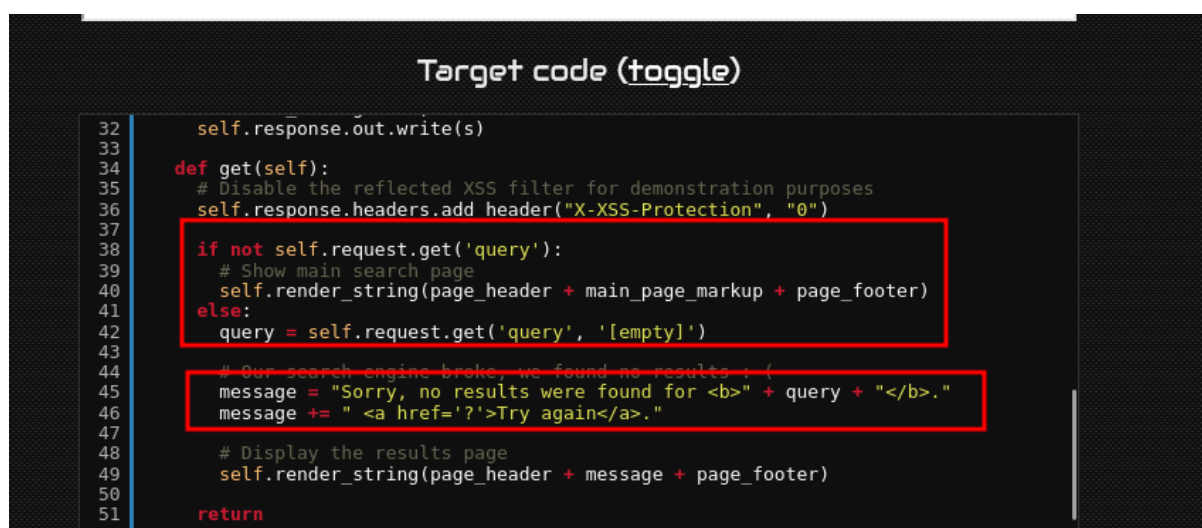


Figura 7. Código vulnerable a XSS.

Puede ser vulnerable a introducciones de usuario como “<script>alert(‘XSS’)</script>”. El navegador recibe <script>alert(‘XSS’)</script> y ejecuta el script.



Figura 8. Explotación de la vulnerabilidad XSS.

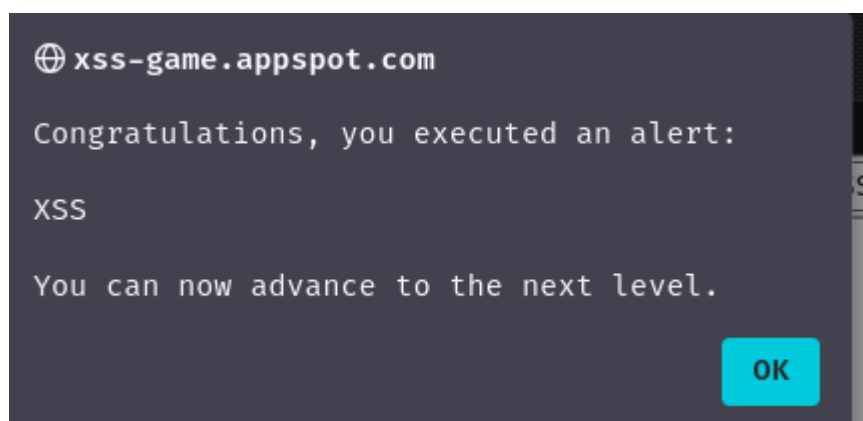


Figura 9. Primer nivel completado.

Level 2 - XSS almacenado en <blockquote>

En el segundo nivel, la aplicación permite a los usuarios publicar mensajes que se almacenan y se muestran posteriormente a todos los visitantes. El fallo de seguridad ocurre porque el contenido del mensaje se inserta directamente en el DOM sin ningún tipo de escape o sanitización. Al estar almacenado en la base de datos (DB), este XSS es de tipo persistente (stored).

El fragmento de código vulnerable es el siguiente:

Target code (toggle)

```

22  var posts = DB.getPosts();
23  for (var i=0; i<posts.length; i++) {
24      var html = '<table class="message"> <tr> <td valign=top> '
25          + ' </td> <td valign=top> '
26          + ' class="message-container"> <div class="shim"></div>';
27
28      html += '<b>You</b>';
29      html += '<span class="date">' + new Date(posts[i].date) + '</span>';
30      html += "&<b>blockquote>" + posts[i].message + "</blockquote>";
31      html += "</td></tr></table>"
32      containerEl.innerHTML += html;
33  }
34  }
35
36  window.onload = function() {
37      document.getElementById('clear-form').onsubmit = function() {
38          DB.clear(function() { displayPosts() });
39          return false;
40      }
41
42      document.getElementById('post-form').onsubmit = function() {

```

Figura 10. Código vulnerable.

Como podemos observar, la variable `posts[i].message`, controlada por el usuario, se concatena directamente dentro del bloque `<blockquote>` sin aplicar ninguna función de escape. Esto permite inyectar etiquetas HTML o código JavaScript arbitrario.

Un atacante introduce en el campo de mensaje el siguiente payload:

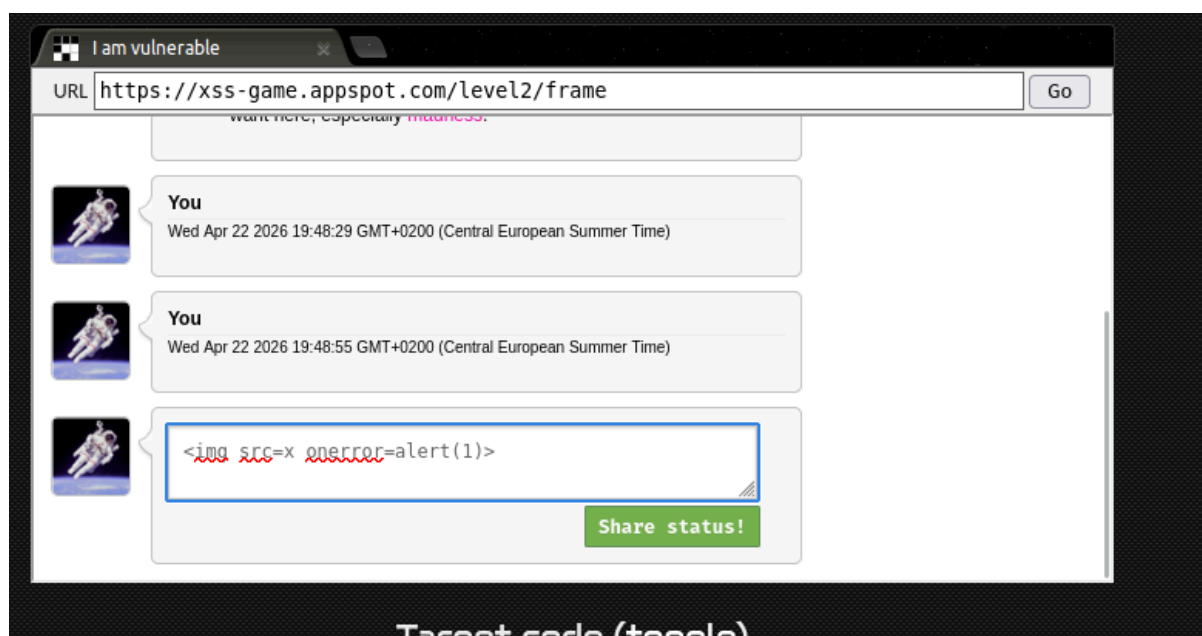


Figura 11. Explotación de la vulnerabilidad.

Cuando otro usuario (o el propio atacante) visualiza el muro de mensajes, el navegador intenta cargar la imagen con la fuente `x`, lo que provoca un error.

Debido al atributo onerror, se ejecuta el código JavaScript alert(1), mostrando la ventana emergente.

Cuando otro usuario (o el propio atacante) visualiza el muro de mensajes, el navegador intenta cargar la imagen con la fuente x, lo que provoca un error. Debido al atributo onerror, se ejecuta el código JavaScript alert(1), mostrando la ventana emergente.

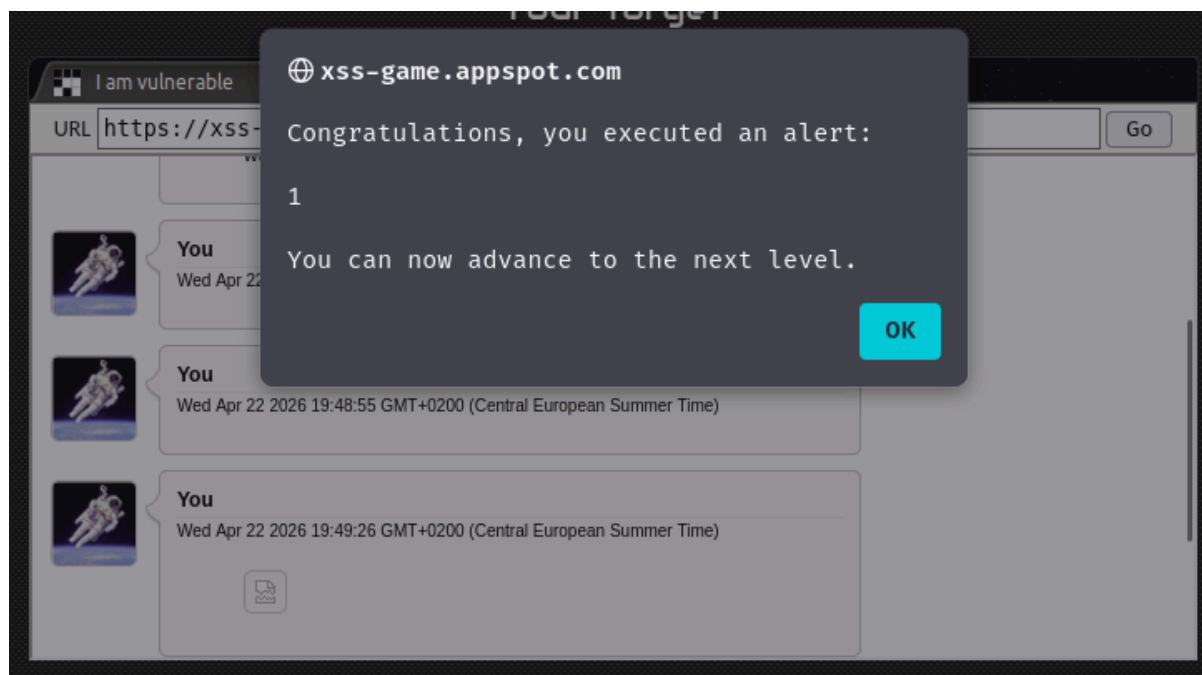


Figura 12. Explotación y laboratorio superado.

Level 3 - XSS basado en el hash de la URL

En el tercer nivel, la aplicación muestra diferentes imágenes de un "cloud data center" según el número de pestañas seleccionadas. El parámetro se pasa a través del hash de la URL (lo que va después del símbolo #). La vulnerabilidad es de tipo DOM-based XSS, ya que el ataque ocurre completamente en el lado del cliente, sin que el servidor procese el payload malicioso.

El fragmento de código vulnerable es el siguiente:

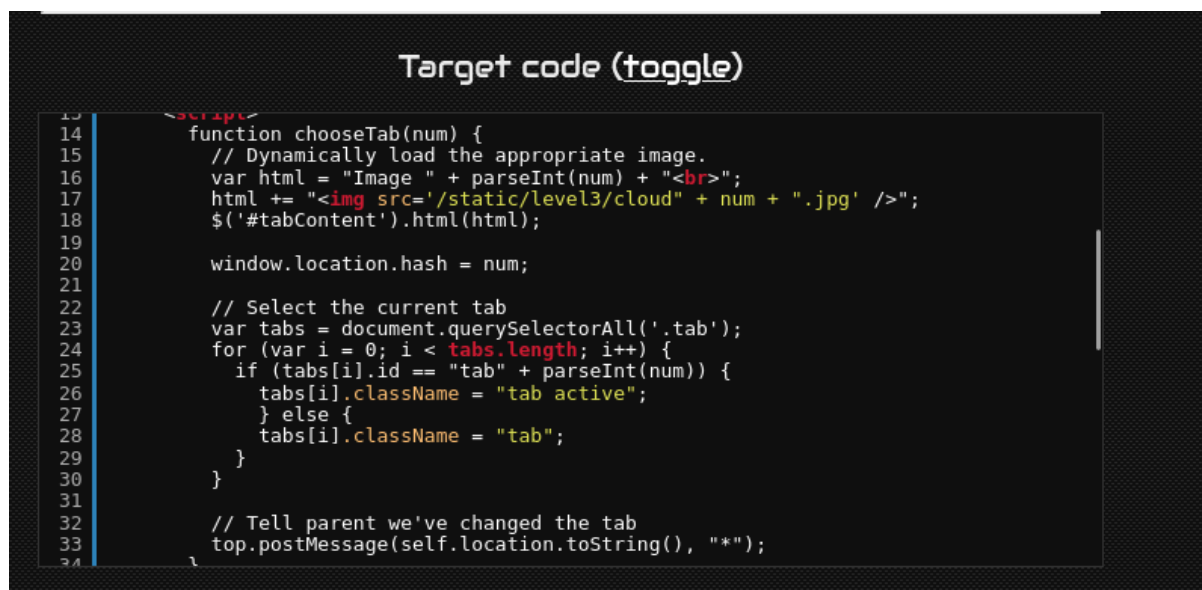


Figura 13. Código vulnerable.

La vulnerabilidad reside en que la variable num, que proviene directamente del hash de la URL (por ejemplo, #1, #2, etc.), se concatena sin sanitización dentro del atributo src de la etiqueta y también en el texto. Un atacante puede cerrar comillas y atributos para inyectar código JavaScript.

Se introduce el siguiente payload en el hash de la URL: `https://xss-game.appspot.com/level3/frame#' onerror='alert(1)' src='x`.

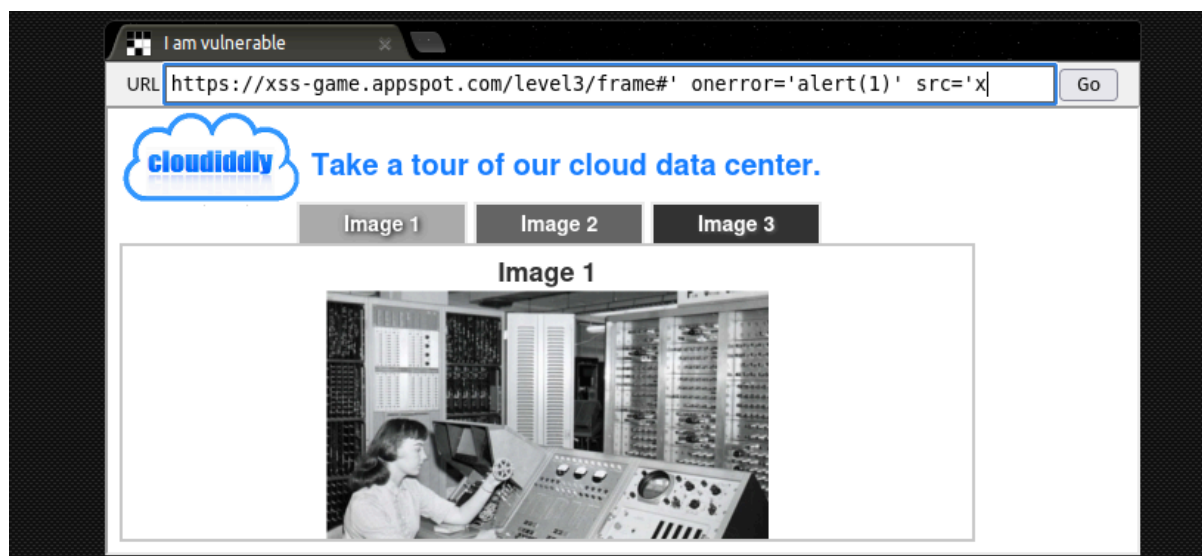


Figura 14. Explotación de la vulnerabilidad.

Cuando la función chooseTab() construye la cadena HTML, el resultado es: `<img src='/static/level3/cloud#' onerror='alert(1)' src='x.jpg' />`.

El navegador interpreta lo siguiente:

- El atributo src se cierra en cloud# (la # es válida en el contexto de URL).
- Aparece un nuevo atributo onerror='alert(1)' que el navegador ejecutará.
- Finalmente otro src='x.jpg' que intenta cargar una imagen inexistente, disparando onerror.

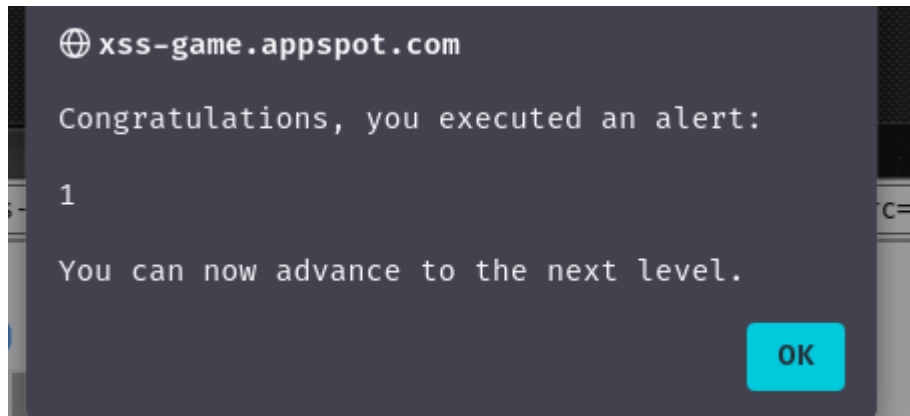


Figura 15. Reto superado.

Este tipo de XSS es particularmente sutil porque el payload nunca viaja al servidor; se procesa íntegramente en el cliente. Para prevenirlo, nunca se debe confiar en datos de window.location (hash, search, etc.) para construir HTML dinámico. En su lugar, se deben usar métodos seguros como textContent, innerText, o funciones de escape específicas.

Level 4 – XSS a través del parámetro de consulta con timer

En el cuarto nivel, la aplicación muestra un temporizador que ejecuta una acción después de un número determinado de segundos. El valor de los segundos se toma del parámetro timer en la URL (?timer=3). Este valor se inserta directamente en dos lugares: dentro del evento onload de una imagen y dentro del texto visible de la página.

El código vulnerable se encuentra en la carga de la imagen:

```
        window.history.back();
    }, seconds * 1000);
}
</script>
</head>
<body id="level4">
    
    <br>
    
    <br>
    <div id="message">Your timer will execute in {{ timer }} seconds.</div>
</body>
</html>
```

Figura 16. Código vulnerable.

La variable `{{ timer }}` es insertada directamente por el servidor (template injection) sin escape, permitiendo al atacante cerrar las comillas simples y ejecutar código JavaScript.

Se construye la siguiente URL:
`https://xss-game.appspot.com/level4/frame?timer=',+alert(1),+`

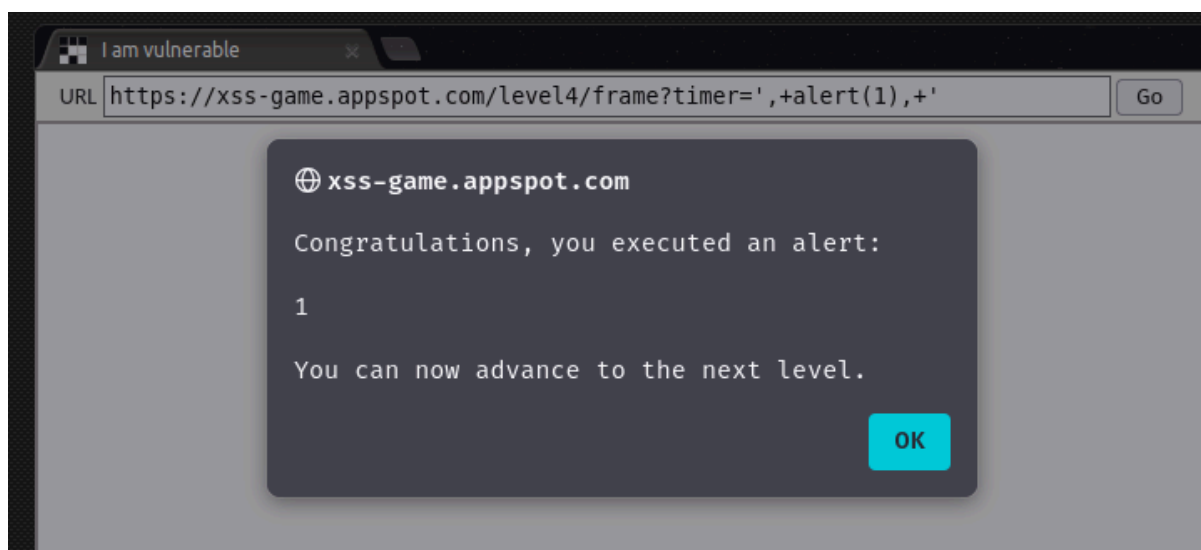


Figura 17. Explotación de la vulnerabilidad.

Este ejemplo muestra cómo la inyección en plantillas (server-side template injection) puede ser igual de peligrosa que el XSS tradicional. La mitigación principal es escapar siempre los datos que se insertan en atributos HTML y eventos JavaScript, no solo en el cuerpo del HTML. Una buena práctica es usar funciones como `escape()` o `encodeURIComponent()` según el contexto.

Level 5 – XSS a través de enlace en parámetro "next"

En el quinto nivel, la aplicación muestra un formulario de suscripción con un enlace "Next >" que redirige al usuario a la URL especificada en el parámetro `?next=`. Este tipo de funcionalidad es común para redirecciones post-login o post-registro. Si el parámetro no se valida adecuadamente, un atacante puede inyectar un protocolo javascript: para ejecutar código arbitrario.

El código HTML generado es: `<a href="{{ next }}">Next></a>`.



```

Target code (toggle)

confirm.html level.py signup.html welcome.html

1 <!doctype html>
2 <html>
3   <head>
4     <!-- Internal game scripts/styles, mostly boring stuff -->
5     <script src="/static/game-frame.js"></script>
6     <link rel="stylesheet" href="/static/game-frame-styles.css" />
7   </head>
8
9   <body id="level5">
10    <br><br>
11    <!-- We're ignoring the email, but the poor user will never know! -->
12    Enter email: <input id="reader-email" name="email" value="">
13
14    <br><br>
15    <a href="{{ next }}">Next >></a>
16  </body>
17 </html>

```

Figura 18. Código vulnerable.

La variable `{{ next }}` se inserta directamente en el atributo href del enlace sin validación previa. Esto permite usar pseudo-protocolos como javascript:.

Se construye la siguiente URL:
[https://xss-game.appspot.com/level5/frame/signup?next=javascript:alert\(1\)](https://xss-game.appspot.com/level5/frame/signup?next=javascript:alert(1)).

El parámetro next tiene el valor javascript:alert(1).



Figura 19. Explotación de la vulnerabilidad.

Cuando el usuario hace clic en el enlace "Next >", el navegador interpreta `javascript:alert(1)` como una pseudo-URL y ejecuta el código JavaScript contenido en ella. En este caso, muestra el `alert(1)`. Alternativamente, el atacante puede introducir directamente `javascript:alert(1)` en el campo de email, y el enlace lo tomará como destino.

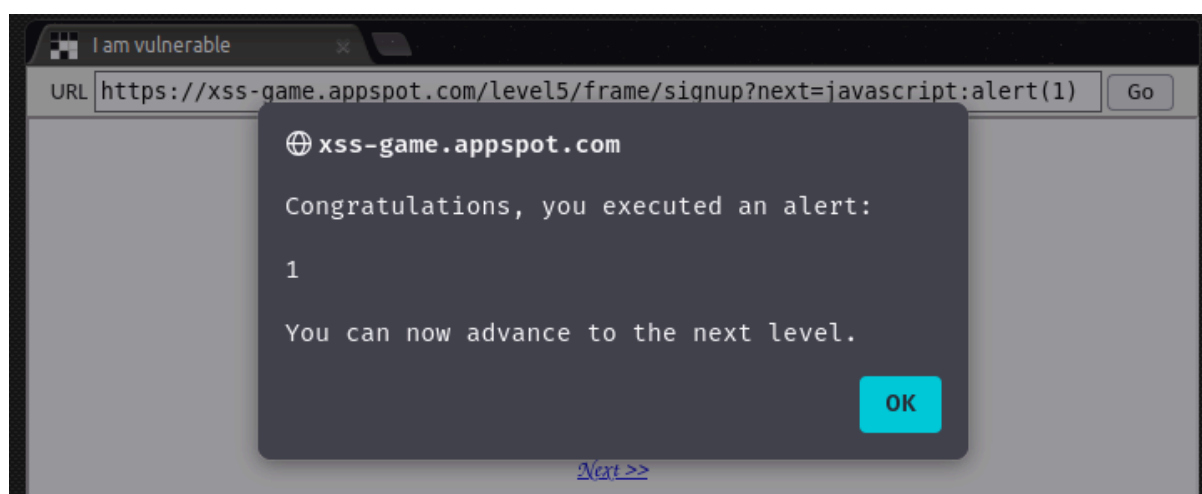


Figura 20. Laboratorio superado.

Este vector de ataque se conoce como "javascript: pseudo-protocol XSS" y es especialmente peligroso porque el usuario solo necesita hacer clic en un enlace aparentemente legítimo. Para mitigarlo:

- Nunca permitir `javascript:` en URLs, ni siquiera después de validar el dominio.

- Validar que la URL comience con un esquema seguro (http:// o https://) y pertenezca a un dominio permitido (whitelist).
- Usar sanitize-url o librerías similares para limpiar URLs peligrosas.

Level 6 – XSS a través de esquemas de URL no estándar

El sexto nivel es el más avanzado. La aplicación carga "gadgets" (scripts) dinámicamente a través de una función includeGadget(url). Se implementa una validación que solo bloquea URLs que contengan "http" (minúsculas), pero no se tienen en cuenta otros esquemas de URL ni variaciones en mayúsculas. Esta validación es insuficiente y puede ser evadida de múltiples formas.



Figura 21. Código vulnerable.

El código comprueba si la URL comienza con http:// o https:// usando una expresión regular. Si es así, la bloquea. Sin embargo:

- No bloquea otros esquemas como data:.
- La expresión regular es case-sensitive, por lo que HTTP:// o HTTPS:// en mayúsculas la evaden.
- No verifica el contenido real del script cargado.

Se utiliza el esquema data: para incrustar JavaScript directamente en la URL: `https://xss-game.appspot.com/level6/frame#data:text/javascript,alert(1)`.

El hash contiene data:text/javascript,alert(1). Este esquema no contiene "http", por lo que pasa la validación. El navegador carga el script con `src="data:text/javascript,alert(1)"`, que se ejecuta inmediatamente.



Figura 22. Primer vector para explotar la vulnerabilidad.

Se utiliza HTTPS:// (mayúsculas) para evadir la expresión regular case-sensitive: `https://xss-game.appspot.com/level6/frame#HTTPS://www.google.com/jsapi?callback=alert`. La URL contiene HTTPS:// con mayúsculas. La expresión regular `/^https?:\W//` busca minúsculas, por lo que no la bloquea. El script se carga y se ejecuta, posiblemente explotando callbacks como `?callback=alert`.



Figura 23. Segundo vector para explotar la vulnerabilidad.

Este nivel demuestra que las validaciones basadas en blacklists (listas negras) son insuficientes y propensas a evasiones. En lugar de bloquear "http", lo correcto es usar una whitelist (lista blanca) de esquemas permitidos (solo `http://` y `https://`, en cualquier combinación de mayúsculas/minúsculas normalizada). Además, se debe validar que el dominio de destino sea de confianza.

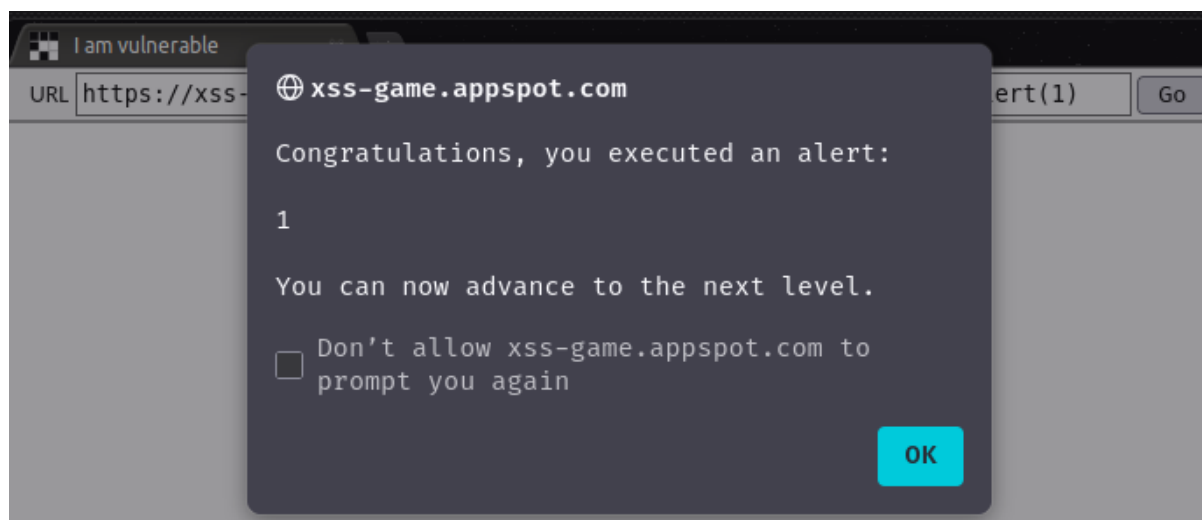


Figura 24. Laboratorio conseguido.

Todos los niveles completados

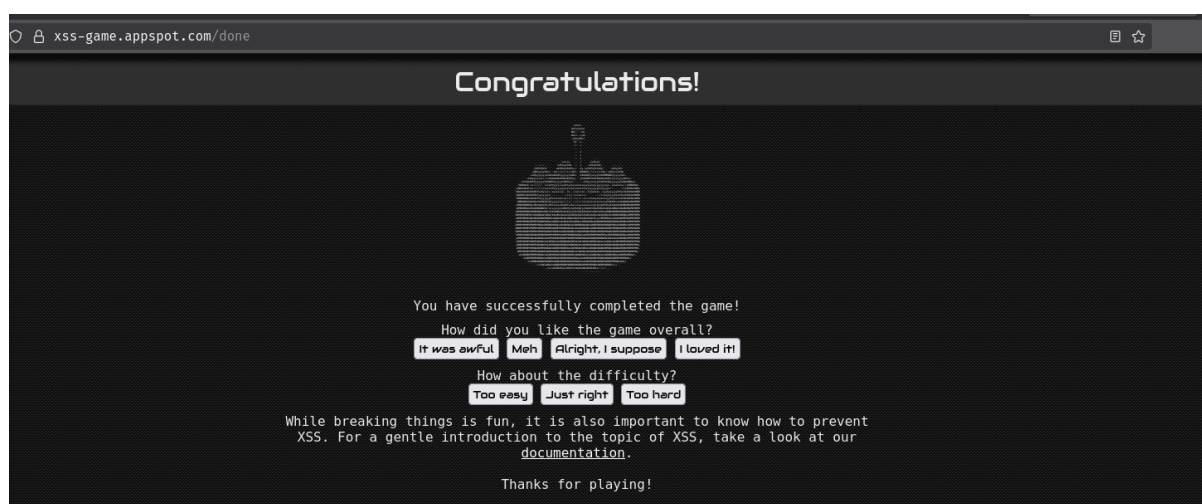


Figura 25. Niveles completados.

3. Funcionamiento del DOM Based XSS

El DOM Based XSS ocurre cuando la vulnerabilidad está completamente en el lado del cliente, es decir, en el código JavaScript que se ejecuta en el navegador. El servidor no tiene ninguna culpa porque devuelve una página perfectamente limpia y segura. El problema surge cuando el JavaScript del cliente lee datos de fuentes controlables por el atacante (como la URL, el hash, localStorage, etc.) y los escribe directamente en el DOM (Document Object Model) sin sanitizarlos.

La aplicación muestra un visualizador de imágenes de un "cloud data center". El usuario puede hacer clic en las pestañas "Image 1", "Image 2" o "Image 3". Al hacer clic, la URL cambia su hash (lo que va después del #) al número de la imagen: #1, #2 o #3. El código JavaScript lee ese hash y construye dinámicamente la etiqueta correspondiente.

Código posible vulnerable:

```
function chooseTab(num) {  
    // num = valor extraído del hash de la URL (ej: "1", "2", "3")  
    var html = "Image" + parseInt(num) + "<br>";  
    html += "<img src='/static/level3/cloud' + num + '.jpg' />";  
    $('#tabContent').html(html); // ¡Aquí está la vulnerabilidad!  
  
    window.location.hash = num;  
}
```

La variable num proviene directamente del hash de la URL (controlable por el atacante) y se concatena sin ningún tipo de escape dentro del atributo src de la etiqueta . Luego se inyecta en el DOM mediante \$('#tabContent').html(html).

Paso 1: El atacante construye una URL maliciosa

En lugar de un número normal como #1 o #2, el atacante construye la siguiente URL: `https://xss-game.appspot.com/level3/frame#'onerror='alert(1)'` `src='x`

El hash completo es: `#'onerror='alert(1)'` `src='x`

Paso 2: El atacante engaña a la víctima para que haga clic en el enlace. El atacante puede enviar este enlace por email, redes sociales, o incrustarlo en un sitio web malicioso. La víctima, sin saberlo, hace clic.

Paso 3: El servidor responde con una página limpia. Cuando la víctima accede a la URL, el servidor de `xss-game.appspot.com` devuelve una página HTML completamente normal y segura. El servidor no ve ni procesa el hash (todo lo que va después de # es exclusivamente del lado del cliente). Por lo tanto, el servidor no es vulnerable; su respuesta no contiene ningún payload malicioso.

Paso 4: El navegador carga la página y ejecuta el JavaScript. Una vez que el navegador recibe la página HTML, esta contiene el código vulnerable que hemos visto antes. El navegador ejecuta ese JavaScript y este lee el hash de la URL actual, que es: `#' onerror='alert(1)' src='x`

Paso 5: El código vulnerable construye el HTML malicioso. La función `chooseTab()` recibe como parámetro `num` el valor extraído del hash (literalmente `#' onerror='alert(1)' src='x`). Luego construye la siguiente cadena HTML:

```
var html = "Image" + parseInt(num) + "<br>";  
  
// parseInt('#') devuelve NaN, por lo que queda "ImageNaN<br>"  
  
html += "<img src='/static/level3/cloud'" + num + ".jpg' />";  
  
// Sustituyendo num: "<img src='/static/level3/cloud#' onerror='alert(1)' src='x.jpg' />"
```

El resultado final es:

```
ImageNaN<br>  
<img src='/static/level3/cloud#' onerror='alert(1)' src='x.jpg' />
```

Paso 6: El navegador interpreta el HTML malicioso. El navegador analiza esta etiqueta `<img>` y la interpreta de la siguiente manera:

`src='/static/level3/cloud#'`: Primer atributo `src`. La `#` es válida en una URL de imagen.

`onerror='alert(1)'`: Atributo `onerror` (manejador de errores) que contiene código JavaScript.

`src='x.jpg'`: Segundo atributo `src` (el navegador usa el último que encuentra).

Paso 7: Se ejecuta el código malicioso. El navegador intenta cargar la imagen desde `'x.jpg'` (una imagen que no existe). Al fallar la carga, se dispara el evento `onerror`, que ejecuta el código JavaScript `alert(1)`.

Paso 8: La alerta aparece en la pantalla de la víctima-

El atacante ha conseguido ejecutar JavaScript arbitrario en el navegador de la víctima.